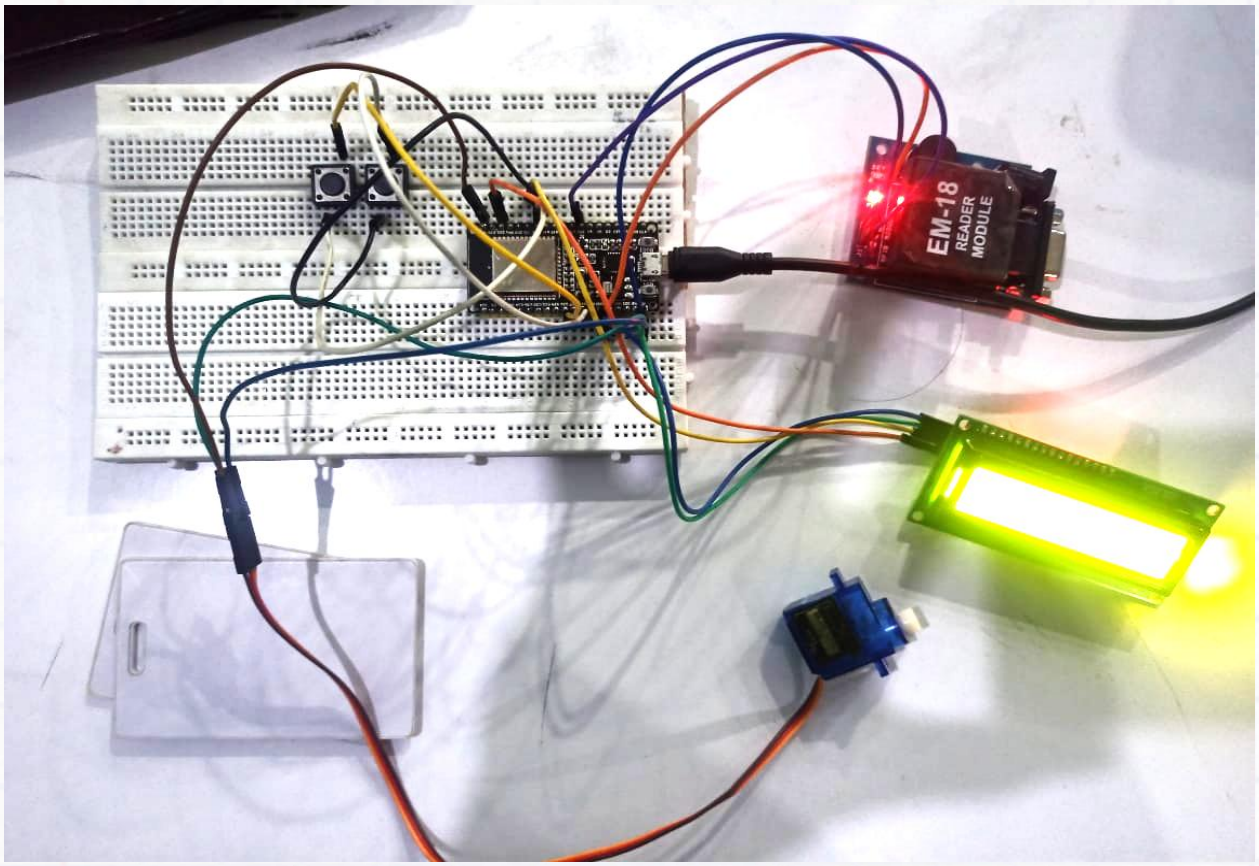
Smart Access Logger (Build a RFID Fast Tag Reader)



Index

Aim

Build an RFID Fast Tag Reader that registers RFID card data, processes it, creates actions, and publishes the data to an online dashboard.

Concept

In this project, we mimic the working of a fast tag reader that reads RFID card data and operates the boom barrier in toll gates. Here we read RFID cards, verify them with the registry in the memory, open/close the gate, and finally send the data to the online dashboard.

Components

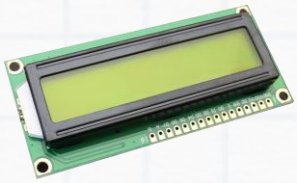
1. ESP 32 C-6 Module
2. RFID Card Reader
3. Servo Motor
4. 16*2 LCD Display
5. Jumper wires
6. USB cables
7. USB to TTL converter (Component to execute extra activities in the project. Procure externally if not available in the kit, CP2102(6-pin) USB 2.0 to TTL UART serial converter)
8. Push button

Esp 32 Module

RFID Reader

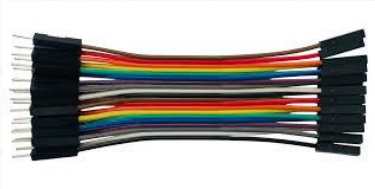
BreadBoard

Servo Motor



16*2 LCD Display

Push button



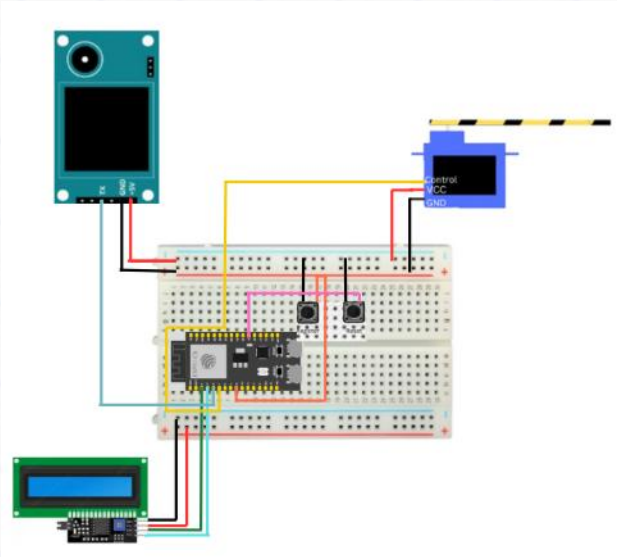
Jumper Wires

USB

Connections

Circuit Diagram

NOTE: Before starting the connections, verify that all the jumper wires are working using a multimeter. Also, ensure that the connections are strong, else the setup may not work.



Detailed Connection Steps

LCD with I2c Module:

- I2c GND to ESP32 GND
- I2c VCC to ESP32 V5
- I2c SDA to ESP32 G4
- I2c SCL to ESP32 G5

Servo

- Servo Out () to ESP32 G7
- Servo 5V to ESP32 V5
- Servo GND to ESP32 GND

Push button

Button1 (Register button):

- Out to ESP32 G8
- GND to ESP32 GND

Button2 (Reset button):

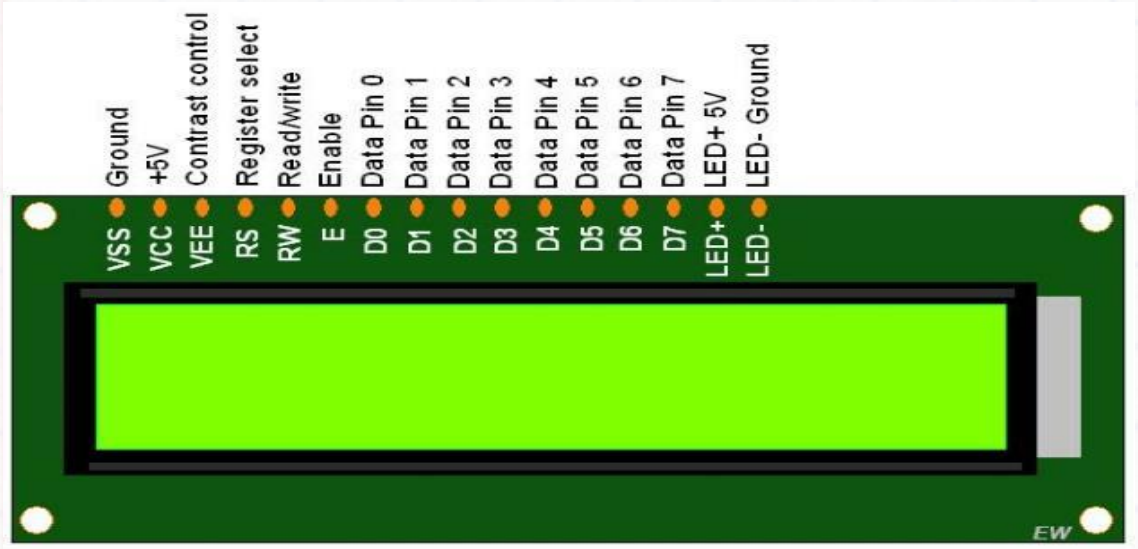
- Out to ESP32 G9
- GND to ESP32 GND

EM-18 Reader Module

- TX to ESP32 G6
- 5v to ESP32 V5
- GND to ESP32 GND

Part 1: Experimenting with the components

1. 16*2 LCD Display



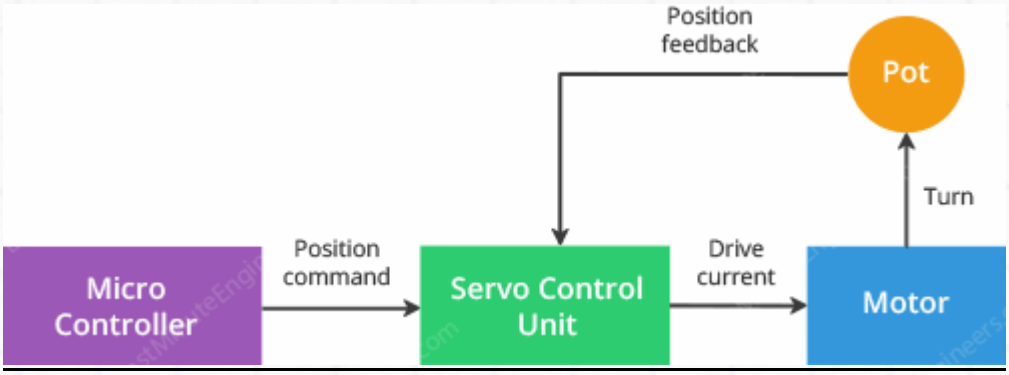
LCD (Liquid Crystal Display) is a flat panel display that uses liquid crystals to display information. It consists of an array of small segments called pixels, which can be manipulated to display status or parameters.

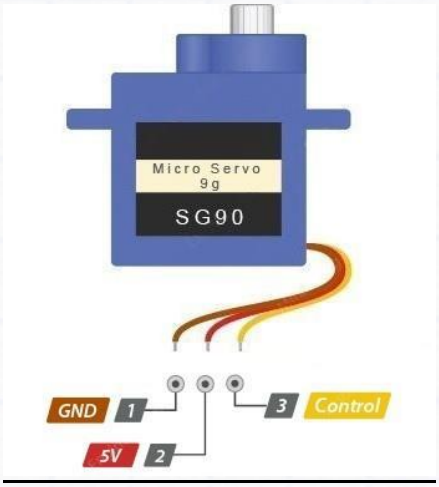
A 16*2 LCD can display 16 characters or numbers column-wise and 2 in row-wise.

2. DC Servo motor

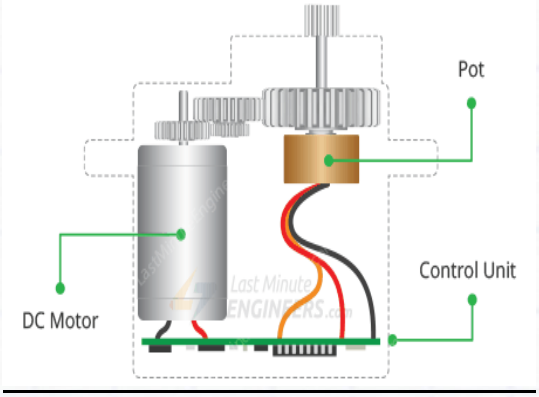
A servo system primarily consists of three basic components: a controlled device, an output sensor, and a feedback system. This is an automatic closed loop control system. Here instead of controlling a device by applying the variable input signal, the device is controlled by a feedback signal generated by comparing output signal and reference input signal. Any electrical motor can be utilised as a servo motor if it is controlled by servomechanism.

In the DC servo motor, a small DC motor connected to the output shaft through the gears. The output shaft drives a servo arm and is also connected to a potentiometer (pot). The potentiometer provides position feedback to the servo control unit where the current position of the motor is compared to the target position. According to the error, the control unit corrects the actual position of the motor so that it matches the target position





DC Servo motor wiring connection



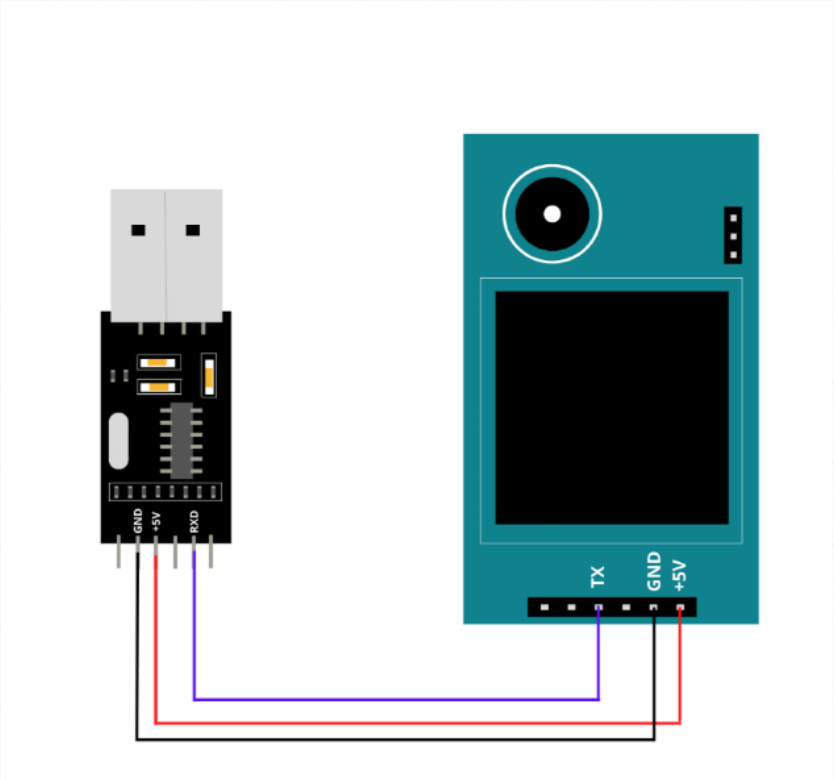
DC Servo motor Internal Illustration

3) RFID Card reader Module

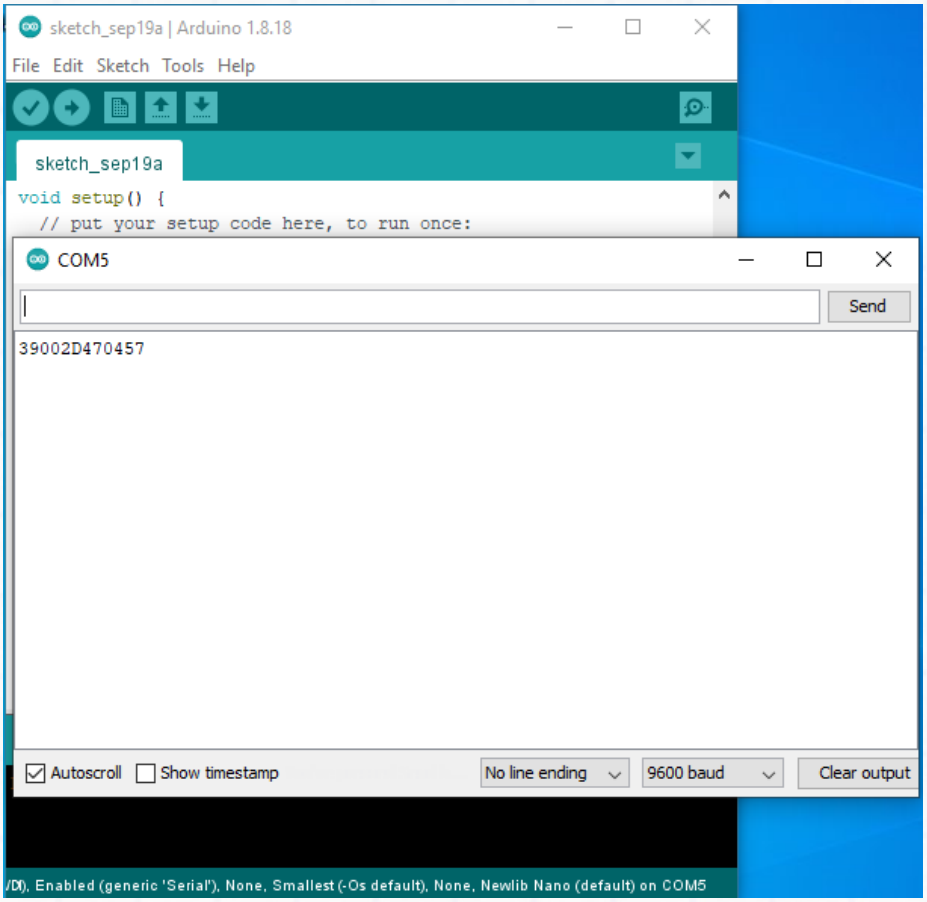
A radio frequency identification reader (RFID reader) module is a device used to gather information from an RFID tag, which is used to track individual tags/cards. Radio waves are used to transfer data from the tag to a reader. RFID is a technology similar to bar codes.

Extra Activity– Reading RFID values

1. Connect a USB to TTL converter to RFID card reader as Below:



2. Show a card to the reader and check the 12 digit RFID codeappearing on Serial Monitor



Note: Each RFID cards will have unique values

Part 2: Implementing the Fast tag reader project

Code:

“Open the .ino file which was attached in the folder and write a code by the below instruction.”

Main Code:

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <ESP32Servo.h>
#include <EEPROM.h>
#include <WiFi.h>
#include <HTTPClient.h>
#include <HardwareSerial.h>
```

```
// Define Pin Mapping for ESP32-C6
#define RFID_RX_PIN 6  // Use GPIO6 for RFID RX (UART1)
#define SERVO_PIN 7
#define REGISTER_BUTTON_PIN 8
#define ERASE_BUTTON_PIN 9

// I2C pins for ESP32-C6
#define I2C_SDA 4
#define I2C_SCL 5

// LCD Configuration
LiquidCrystal_I2C lcd(0x27, 16, 2);
Servo doorServo;

const int MAX_CARDS = 10;
String registeredCards[MAX_CARDS];
int cardCount = 0;
int totalEntries = 0;
int accessGranted = 0;
int accessDenied = 0;

// WiFi settings
const char* ssid = "Mahesh";
const char* password = "123456789";

// ThingSpeak settings
const char* thingSpeakApiKey = "YOUR_THINGSPEAK_APIKEY";
const char* thingSpeakHost = "api.thingspeak.com";

// Define hardware serial port for RFID
HardwareSerial RFIDSerial(1); // Use UART1 for RFID communication

void setup() {
  Serial.begin(115200);
  Serial.println("Starting RFID System...");

  // Initialize RFID Serial (UART1), only RX is used
```

```
  RFIDSerial.begin(115200, SERIAL_8N1, RFID_RX_PIN, -1);

  Wire.begin(I2C_SDA, I2C_SCL);
  lcd.begin();
  lcd.backlight();
  lcd.clear();
  lcd.print("RFID System");
  lcd.setCursor(0, 1);
  lcd.print("Initializing...");

  doorServo.attach(SERVO_PIN);
  doorServo.write(0);

  pinMode(REGISTER_BUTTON_PIN, INPUT_PULLUP);
  pinMode(ERASE_BUTTON_PIN, INPUT_PULLUP);

  EEPROM.begin(512);
  loadCardsFromEEPROM();
  connectWiFi();

  delay(2000);
  lcd.clear();
  lcd.print("System Ready");
  updateThingSpeak();
}

void loop() {
  if (digitalRead(REGISTER_BUTTON_PIN) == LOW) {
    registerCard();
  }

  if (digitalRead(ERASE_BUTTON_PIN) == LOW) {
    eraseAllCards();
  }

  if (RFIDSerial.available()) {
    String cardID = RFIDSerial.readStringUntil('\n');
    cardID.trim();
```

```
    totalEntries++;

    if (isCardRegistered(cardID)) {
      grantAccess();
      accessGranted++;
    } else {
      denyAccess();
      accessDenied++;
    }

    updateThingSpeak();
  }
}

void registerCard() {
  lcd.clear();
  lcd.print("Scan to register");
  while (!RFIDSerial.available()) {
    delay(100);
  }

  String cardID = RFIDSerial.readStringUntil('\n');
  cardID.trim();

  if (cardCount < MAX_CARDS) {
    registeredCards[cardCount++] = cardID;
    saveCardsToEEPROM();

    lcd.clear();
    lcd.print("Card registered");
    lcd.setCursor(0, 1);
    lcd.print(cardID);
    updateThingSpeak();
  } else {
    lcd.clear();
    lcd.print("Max cards reached");
  }
```

```
  delay(2000);
  lcd.clear();
  lcd.print("System Ready");
}

void eraseAllCards() {
  cardCount = 0;
  saveCardsToEEPROM();
  lcd.clear();
  lcd.print("All cards erased");
  updateThingSpeak();
  delay(2000);
  lcd.clear();
  lcd.print("System Ready");
}

bool isCardRegistered(String cardID) {
  for (int i = 0; i < cardCount; i++) {
    if (registeredCards[i] == cardID) {
      return true;
    }
  }
  return false;
}

void grantAccess() {
  lcd.clear();
  lcd.print("Access Granted");
  doorServo.write(90);
  delay(3000);
  doorServo.write(0);
  lcd.clear();
  lcd.print("System Ready");
}

void denyAccess() {
  lcd.clear();
  lcd.print("Access Denied");
```

```
  delay(2000);
  lcd.clear();
  lcd.print("System Ready");
}

void saveCardsToEEPROM() {
  for (int i = 0; i < MAX_CARDS; i++) {
    if (i < cardCount) {
      writeStringToEEPROM(i * 50, registeredCards[i]);
    } else {
      writeStringToEEPROM(i * 50, "");
    }
  }
  EEPROM.commit();
}

void loadCardsFromEEPROM() {
  cardCount = 0;
  for (int i = 0; i < MAX_CARDS; i++) {
    String card = readStringFromEEPROM(i * 50);
    if (card.length() > 0) {
      registeredCards[cardCount++] = card;
    }
  }
}

void writeStringToEEPROM(int addr, String data) {
  int len = data.length();
  EEPROM.write(addr, len);
  for (int i = 0; i < len; i++) {
    EEPROM.write(addr + 1 + i, data[i]);
  }
}

String readStringFromEEPROM(int addr) {
  int len = EEPROM.read(addr);
  String data = "";
  for (int i = 0; i < len; i++) {
```

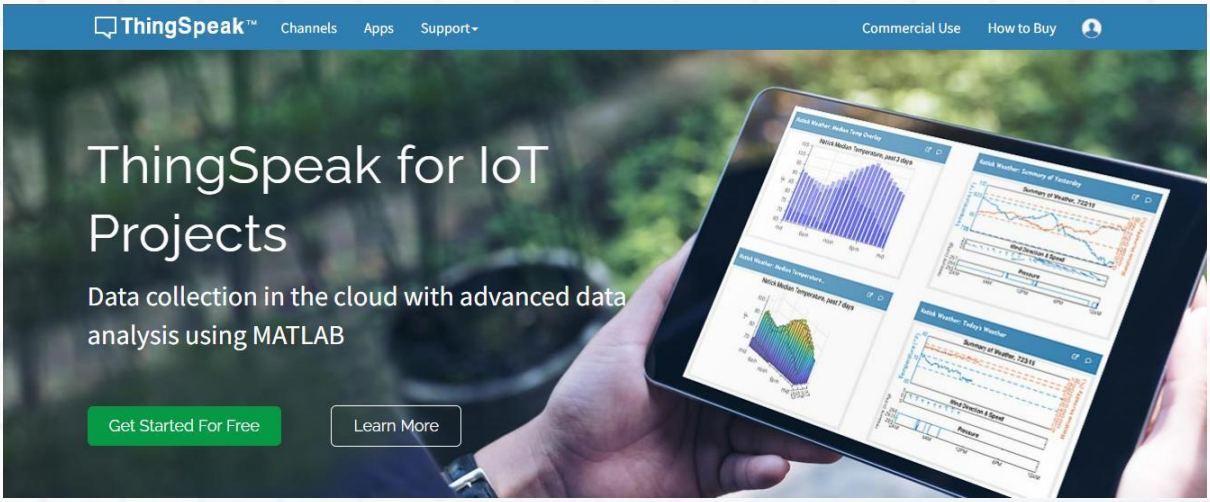
```
    data += char(EEPROM.read(addr + 1 + i));
  }
  return data;
}

void connectWiFi() {
  WiFi.begin(ssid, password);
  lcd.clear();
  lcd.print("Connecting WiFi");

  while (WiFi.status() != WL_CONNECTED) {
    delay(1000);
    lcd.print(".");
  }

  lcd.clear();
  lcd.print("WiFi Connected");
  delay(1000);
}

void updateThingSpeak() {
  if (WiFi.status() != WL_CONNECTED) {
    connectWiFi();
    return;
  }

  HTTPClient http;
  String url = "http://api.thingspeak.com/update?api_key=" +
String(thingSpeakApiKey);
  url += "&field1=" + String(cardCount);
  url += "&field2=" + String(totalEntries);
  url += "&field3=" + String(accessGranted);
  url += "&field4=" + String(accessDenied);

  http.begin(url);
  int httpResponseCode = http.GET();

  http.end();
```

```
  lcd.clear();
  lcd.print("System Ready");
}
```

After completing these all the steps verify and upload a code to the esp32.

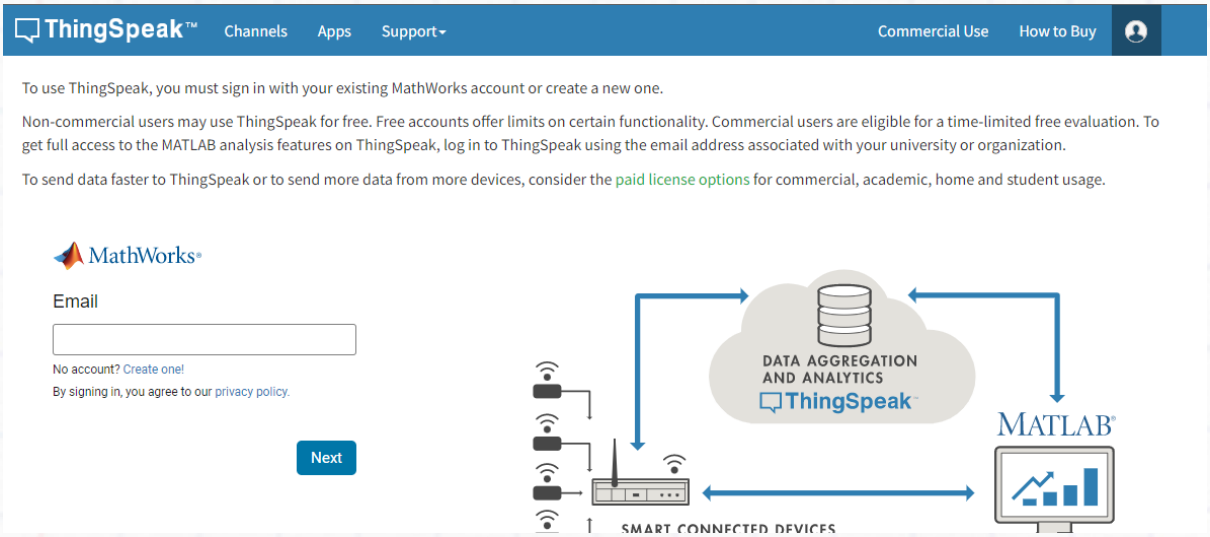
Setting up web dashboard – thingspeak

In previous projects we had used Blynk web platform, this time we will try ThingSpeak. ThingSpeak is an IoT analytics platform service that allows you to aggregate, visualize, and analyse live data streams in the cloud.

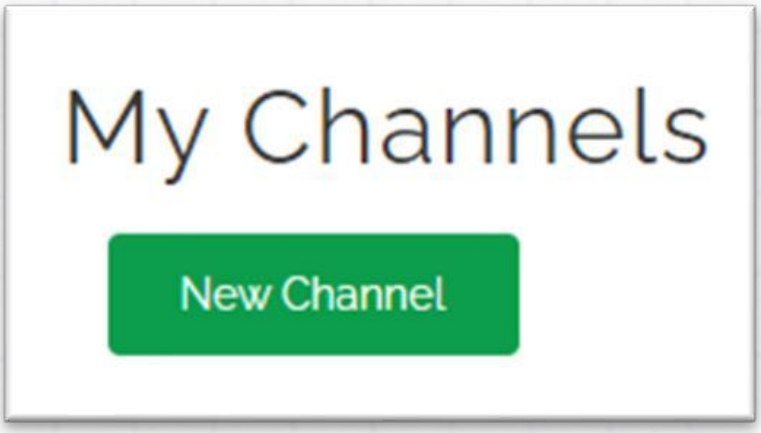


Configuration steps

1. Visit https://thingspeak.com/ website.
2. Create an account



3. Create a new channel in 'My channels'



4. Name the channel as you may prefer like RFID. Tick the fields Field1,Field2,Field3,Field4 since we intend to fill the fields with following data:

- Field1=No of ID's registered
- Field2= Total no. of times card entries
- Field3 = Total no. of access granted
- Field4 = Total no. of access denied

5. Now we have to link this ThingSpeak dashboard with our ESP32 code.

6. Go to '**API keys**' section in RFID channel

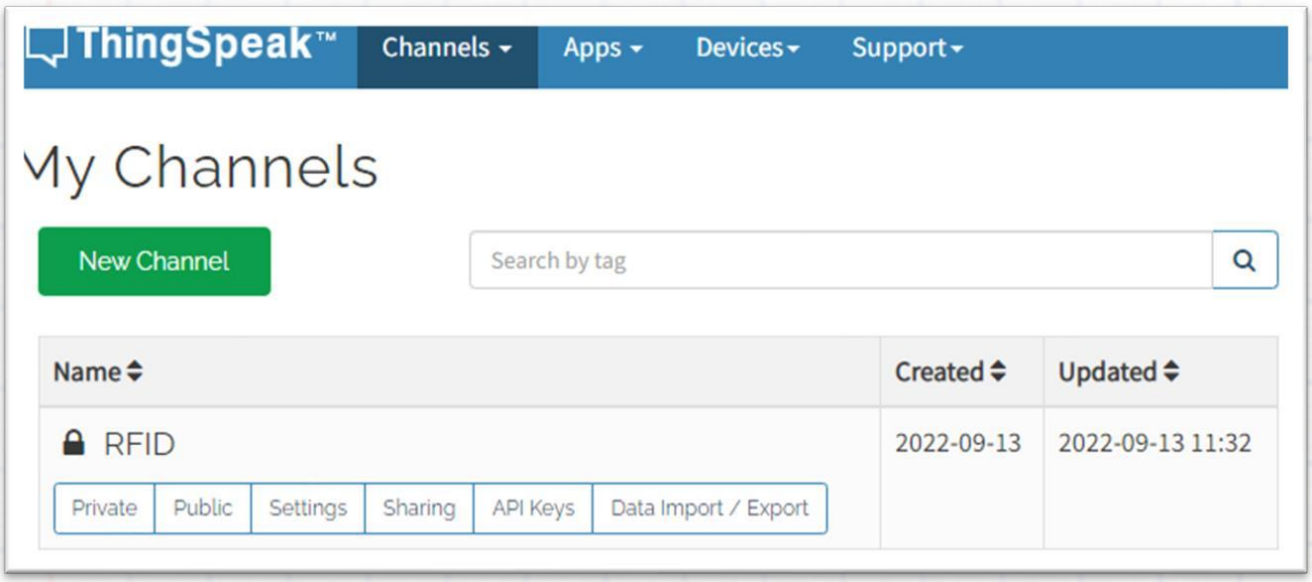
7.Copy the API key and paste it on your code.

```
const char* thingSpeakApiKey = "YOUR_THINKSPEAK_WRITE_API_KEY";
```

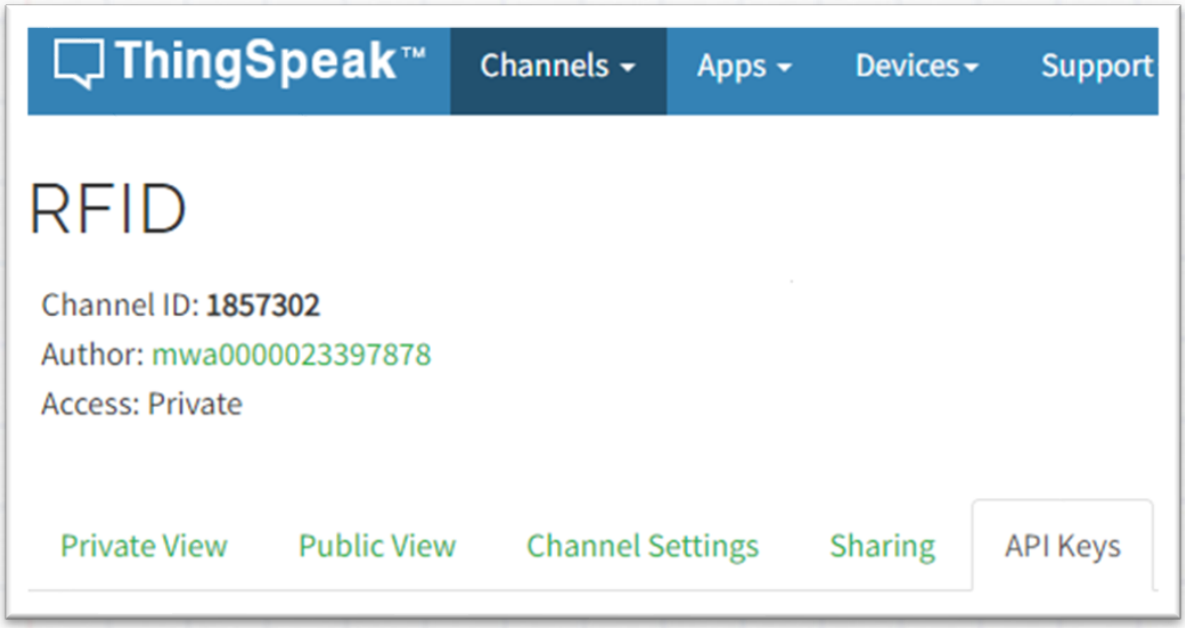
8. Run the code in ESP32 Module

Note: The code run on ESP32 will automatically fill the 4 fields with data

9. Now go to your ‘RFID’ channel dashboard



10.Click RFID channel and navigate to ‘private view’

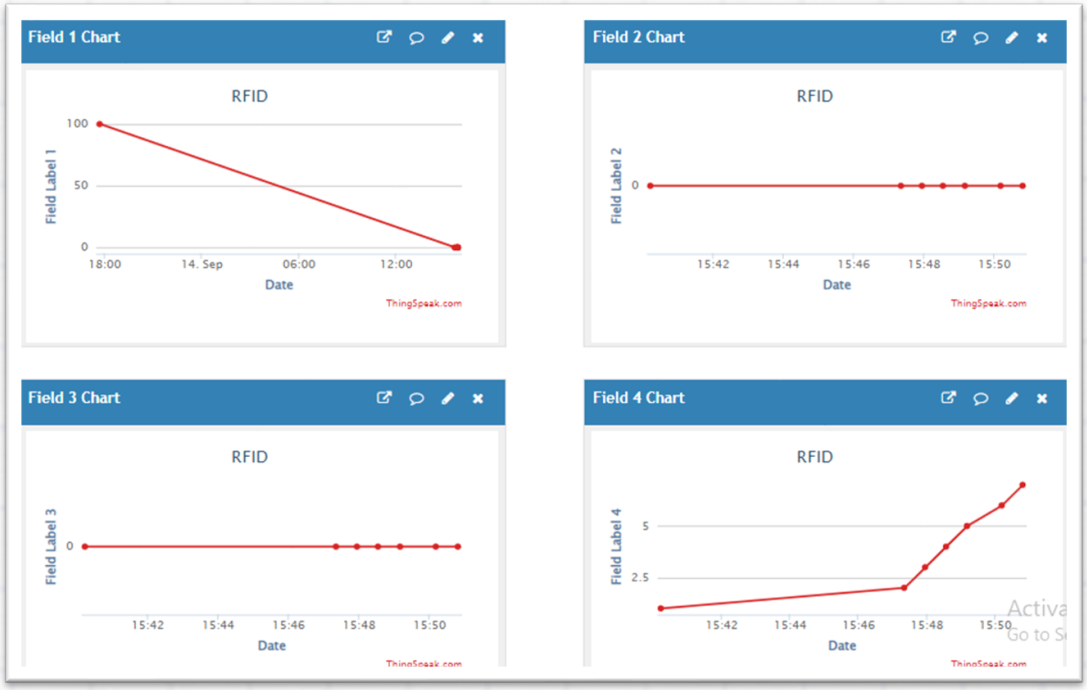


11.Now it’s time to receive data to the web dashboard.

12. Check whether all other components like RFID reader, LCD display servo motor, ESP32 Module are powered up properly.

13.Try using different RFID cards, register a card. Check whether the servomotor is working, and entry is permitted. Try with non-registered cards, check whether entry is denied.

14.Now visit the ThingSpeak dashboard. You will see the fields are getting populated.



Task

1. After erasing the current memory. Take 3 new RFID cards, add 2 cards. Keep one card unregistered. Check how the device is behaving with the 3 cards. Check whether the data is received on the Web dashboard.

2. Make a video presenting the build of RFID reader, how you completed the activities and challenges. Share it with the Build Club Community on the Discord Server!